TRACE-HEP

Comprehensive Documentation - v0.1.8 - Toolkit for Rendering and Analysis of Collider Events

What is TRACE-HEP?

* A format-agnostic Python library for reconstructed-level collider event and vertex displays
* Polar (phi, |eta|), beam-axis 2D, and interactive 3D event views
* A companion per-vertex / all-vertices family for pileup diagnostics
* Core package has zero ROOT/uproot dependency
* Optional loaders for Delphes, flat-ntuple, calo-timing, and public
* ATLAS Open Data ROOT files
* Open source, MIT licensed

Where to find it

```
GitHub:    github.com/wasikatcern/TRACE-HEP
TestPyPI:  test.pypi.org/project/trace-hep
License:   MIT
```

Installation: core package

```
python3 -m venv .venv
source .venv/bin/activate
```

```
pip install --index-url https://test.pypi.org/simple/ \
            --extra-index-url https://pypi.org/simple/ \
                              trace-hep
```

Core install pulls only numpy, matplotlib, and plotly.

Installation: optional extras

```
# ROOT-file loaders: Delphes, flat ntuple, calo-timing
pip install --index-url https://test.pypi.org/simple/ \
            --extra-index-url https://pypi.org/simple/ \
                        "trace-hep[delphes]"

# + static image export (kaleido) and ATLAS style (mplhep)
pip install ... "trace-hep[all]"

# from source instead, editable (for developing tracehep itself)
git clone https://github.com/wasikatcern/TRACE-HEP.git
cd TRACE-HEP
pip install -e ".[dev]"
pytest -q
```

Verify the install

```
python3 -c "import tracehep as trace; print(trace.__version__)"
# -> 0.1.8

trace-batch --help
# -> usage: trace-batch [-h] --input FILE.root --indices N [N ...] ...
```

The data model: one hard-scatter event

```
    Jet(pt, eta, phi, mass=0, btag=False, is_hs=None)
Track(pt, eta, phi, charge=0, d0=0, z0=0, x=0, y=0, z=0,
                        time=None, is_hs=None)
   Lepton(pt, eta, phi, flavor="muon"|"electron")
                  Photon(pt, eta, phi)
                MissingET(pt, phi, eta=0)

Event(jets, tracks, muons, electrons, photons, met,
            run=None, event_number=None, label="")
```

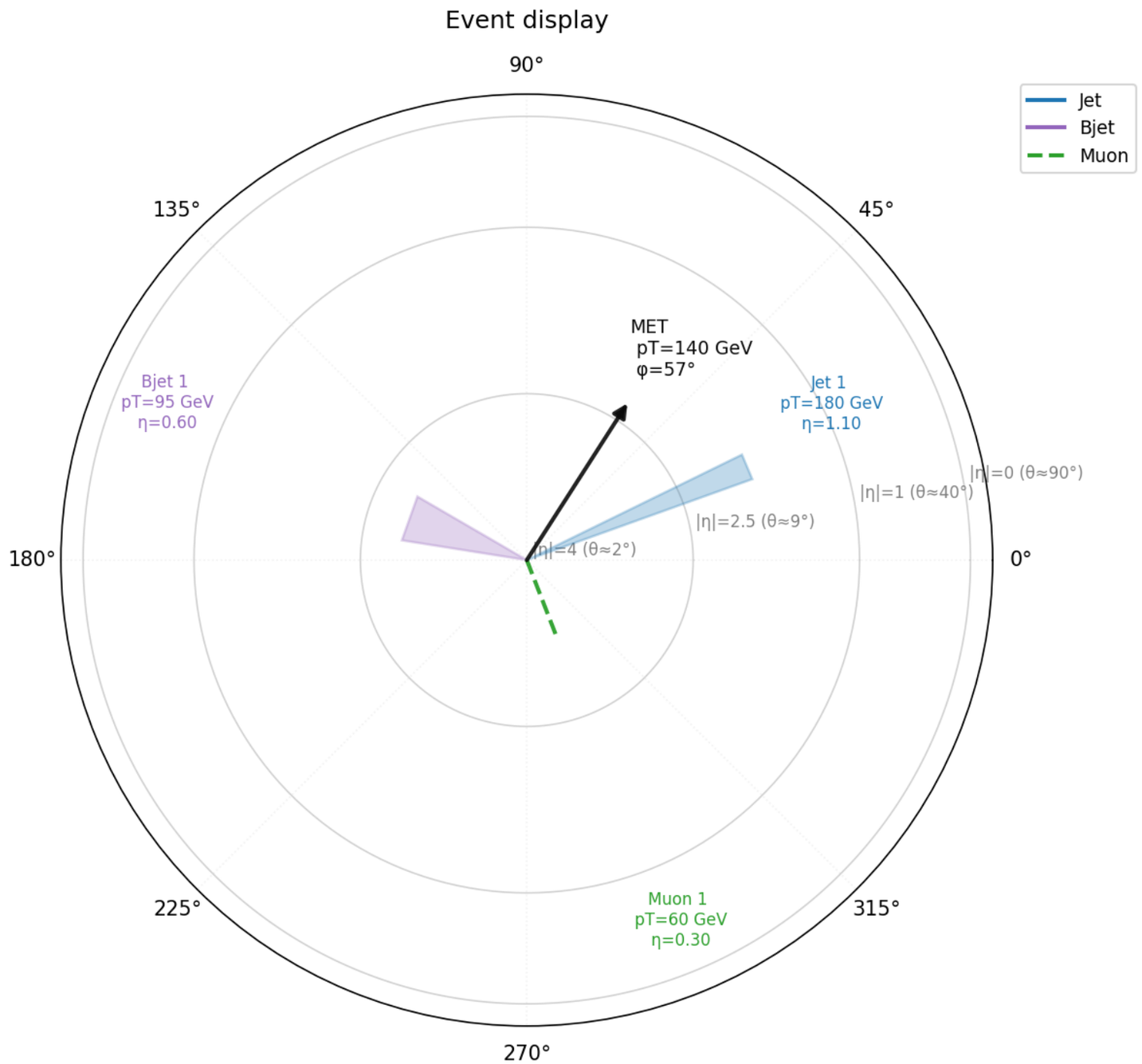
The data model: one pileup scenario

```
Vertex(z, x=0, y=0, sum_pt2=0, is_hs=False, is_pu=False,
                    time=None, track_indices=[])
          TruthVertex(z, x=0, y=0, is_hs=False)

VertexEvent(vertices, tracks, truth_vertices, mu=None, label="")

Nothing here knows about ROOT or any ntuple format -- build these
    by hand from your own analysis objects and every drawing
              function in the package works unchanged.
```

Quickstart: build an event by hand

```
import tracehep as trace

event = trace.Event(
jets=[trace.Jet(pt=180, eta=1.1, phi=0.4),
trace.Jet(pt=95, eta=-0.6, phi=2.8, btag=True)],
muons=[trace.Lepton(pt=60, eta=0.3, phi=-1.2, flavor="muon")],
met=trace.MissingET(pt=140, phi=1.0),
)

fig = trace.plot_event_polar(event)
fig.savefig("event_polar.png")
```

Output: the polar (phi, |eta|) view



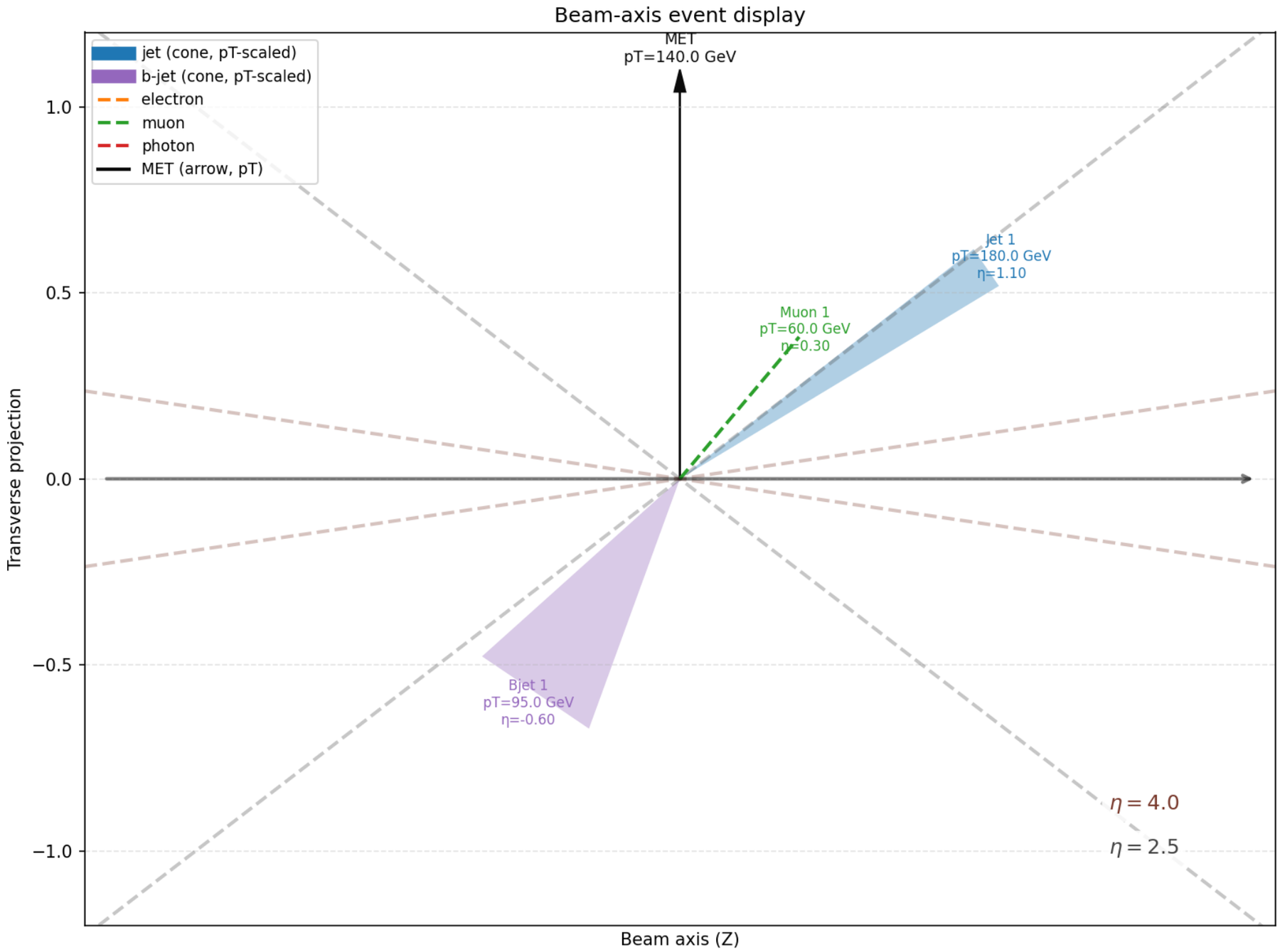
trace.plot_event_polar(event)

The beam-axis 2D projection

```
fig = trace.plot_event_beam2d(event)
  fig.savefig("event_beam2d.png")
```

Complementary side-on view: beam direction horizontal, transverse coordinate vertical.

Output: the beam-axis 2D view

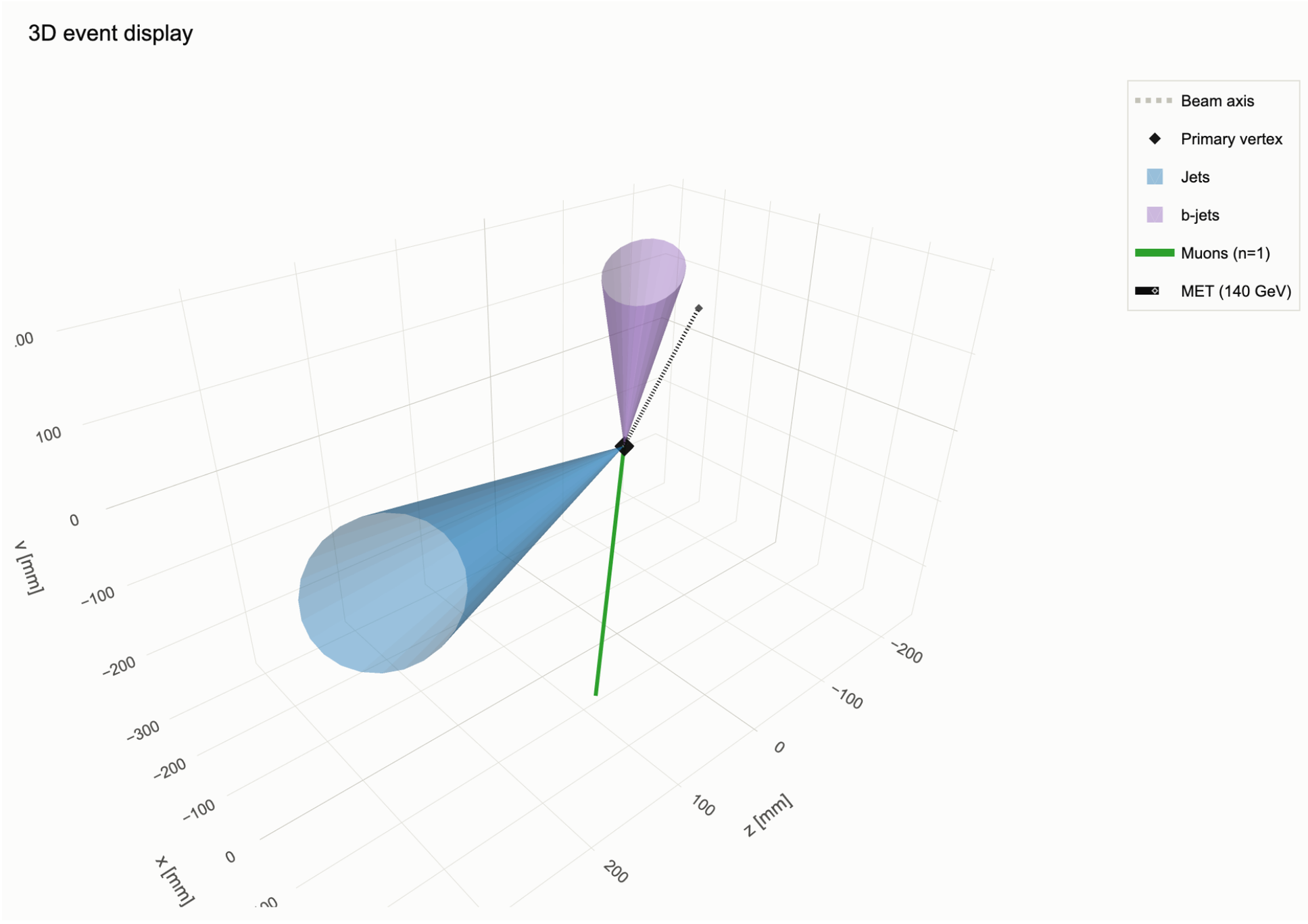


trace.plot_event_beam2d(event)

The interactive 3D view

```
fig3d = trace.plot_event_3d(event)
fig3d.write_html("event_3d.html")

# static snapshot (needs the kaleido extra)
fig3d.write_image("event_3d.png")
```

Output: the interactive 3D view



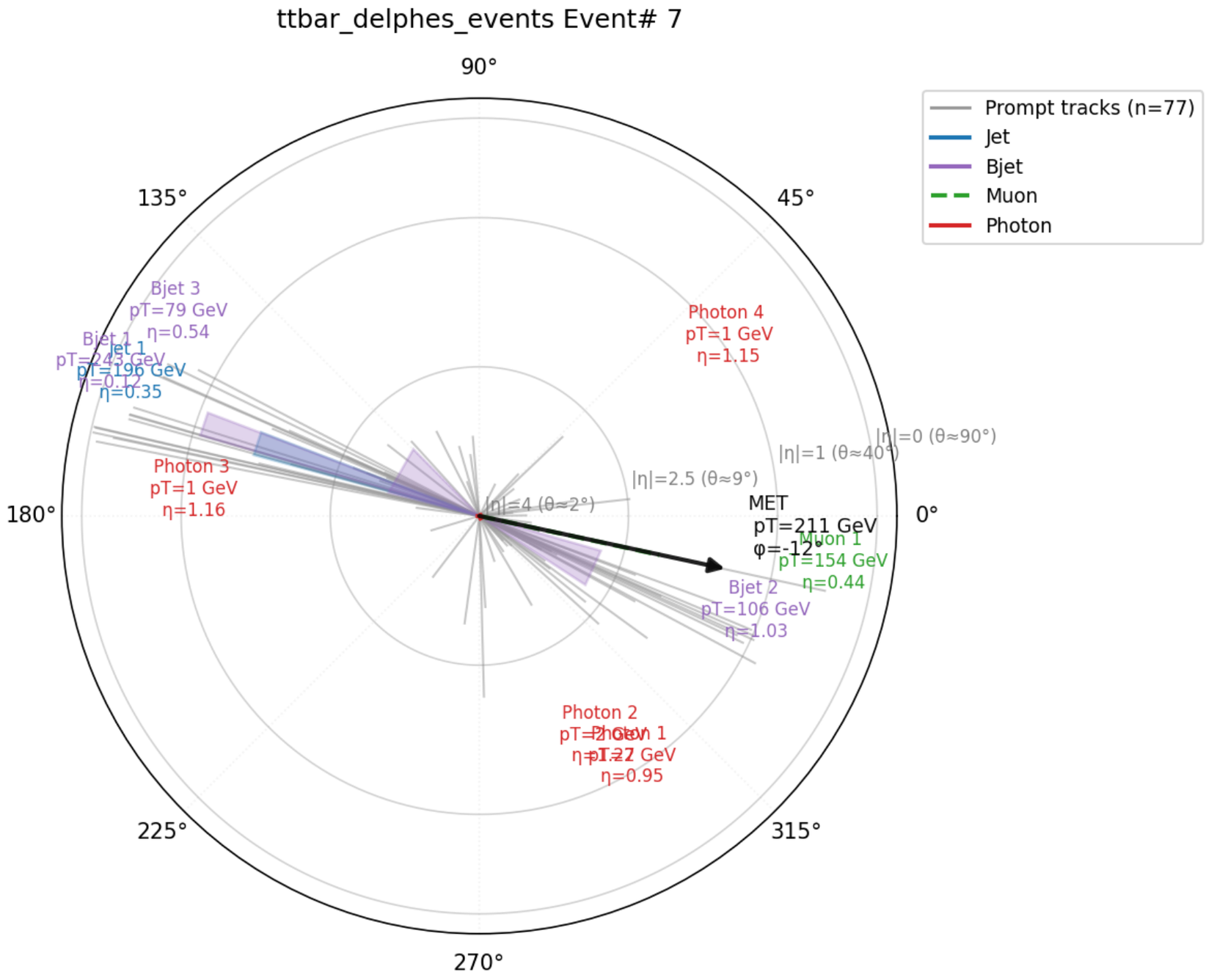
Static snapshot of the interactive HTML (drag to orbit, scroll to zoom)

Loading real events: Delphes ROOT files

```
from tracehep.io.delphes import load_events

events = load_events(
"ttbar_delphes_events.root",
indices=[7],
label="ttbar_delphes_events",
)
event = events[7]

fig = trace.plot_event_polar(event, show_tracks=True)
fig.savefig("event7_polar.png")
```

Needs the [delphes] extra. Works unchanged on any Delphes sample, any physics process.

Output: a real ttbar Delphes event



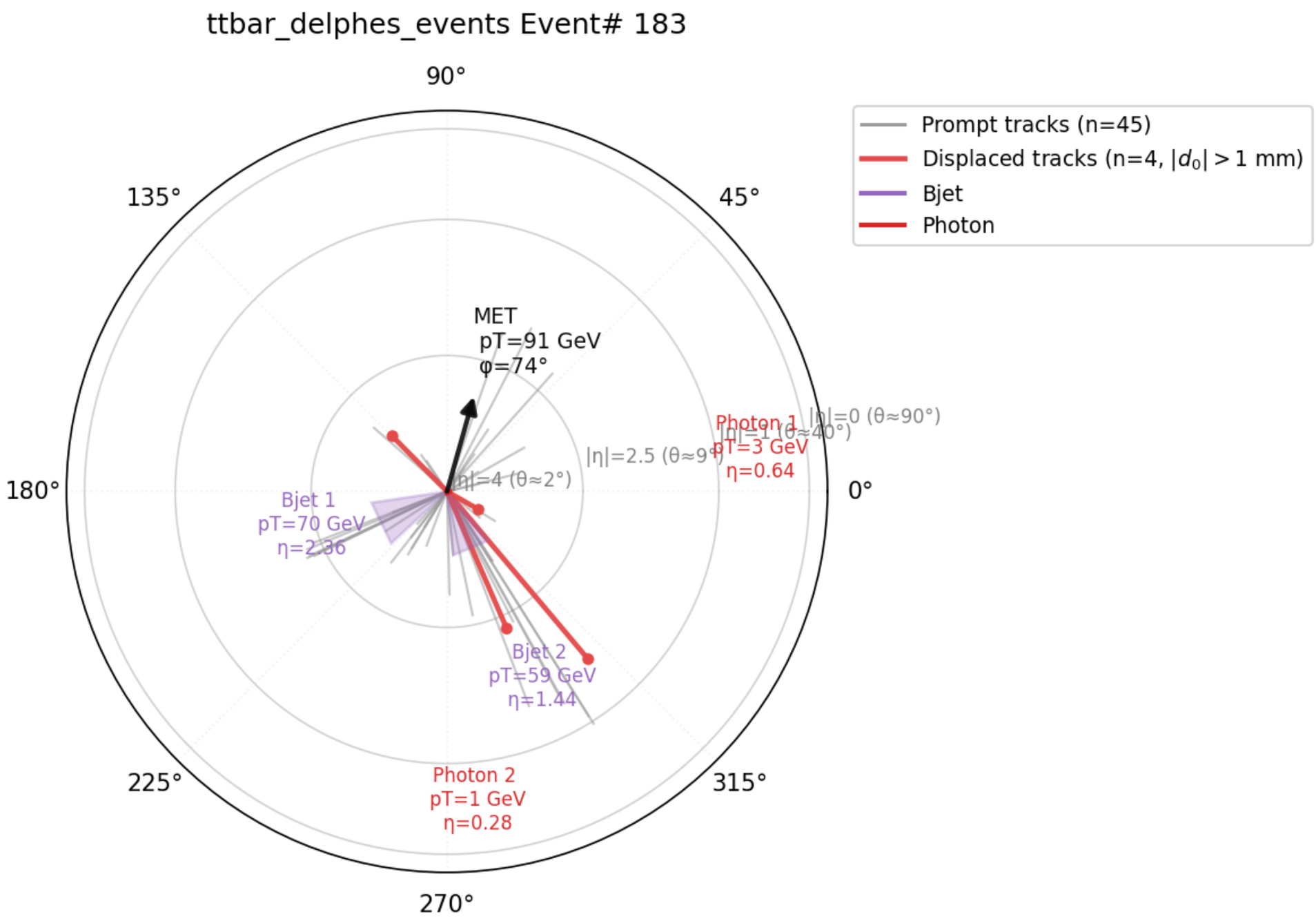
Event #7: 4 jets, 77 tracks, 1 muon, 4 photons, MET = 211 GeV

Highlighting displaced tracks (d0)

```
fig = trace.plot_event_polar(
event, show_tracks=True, d0_displaced_mm=1.0,
)
fig.savefig("event_displaced_polar.png")

fig3d = trace.plot_event_3d(event, d0_displaced_mm=1.0)
fig3d.write_html("event_displaced_3d.html")
```

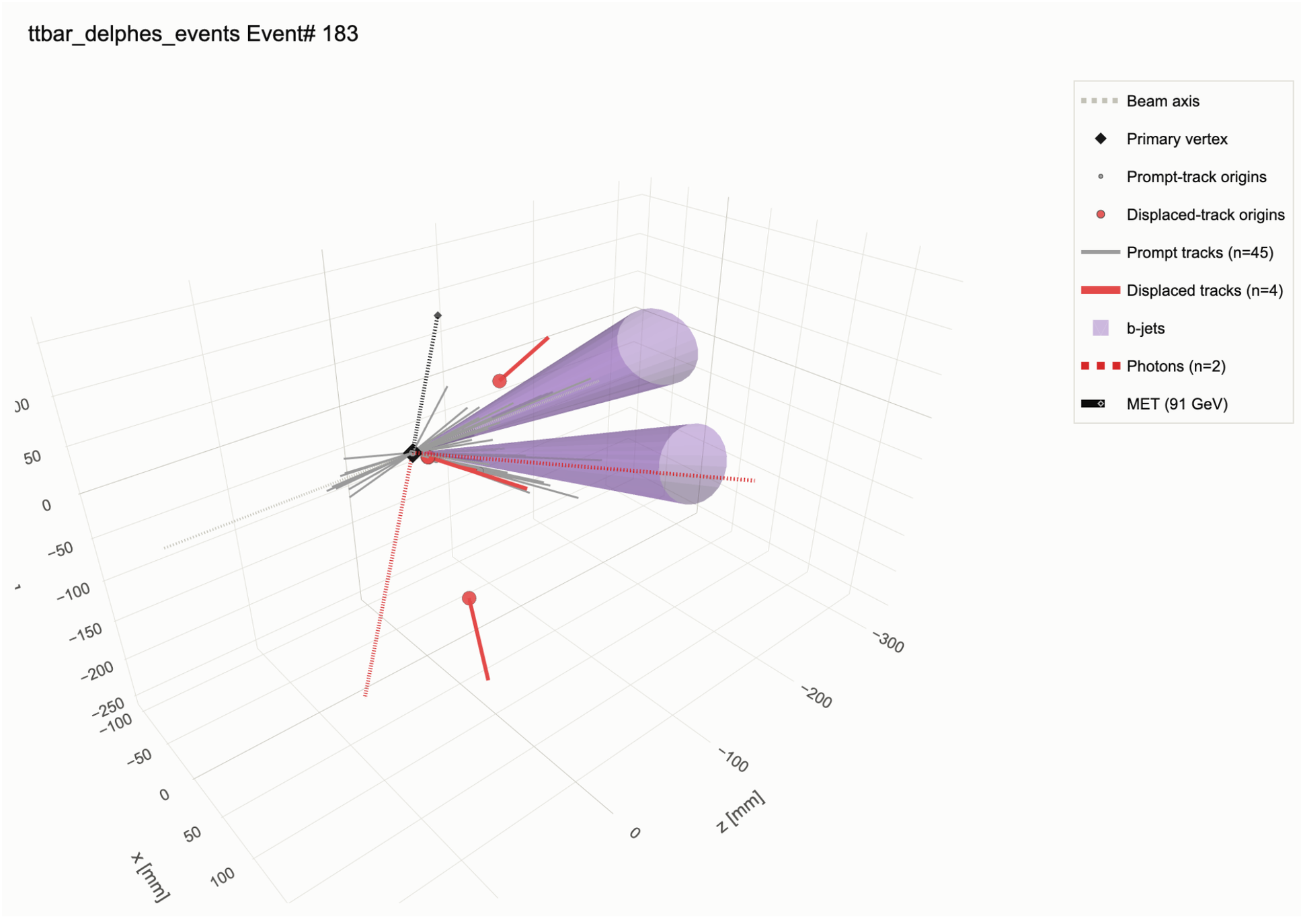
Tracks with |d0| above the threshold are coloured distinctly from prompt tracks.

Output: displaced tracks (polar view)



4 of 45 tracks flagged as displaced (|d0| > 1 mm), picked out in red

Output: displaced tracks (3D view)



Same event -- displaced-track origins marked distinctly in 3D

Vertex displays: loading a pileup ntuple

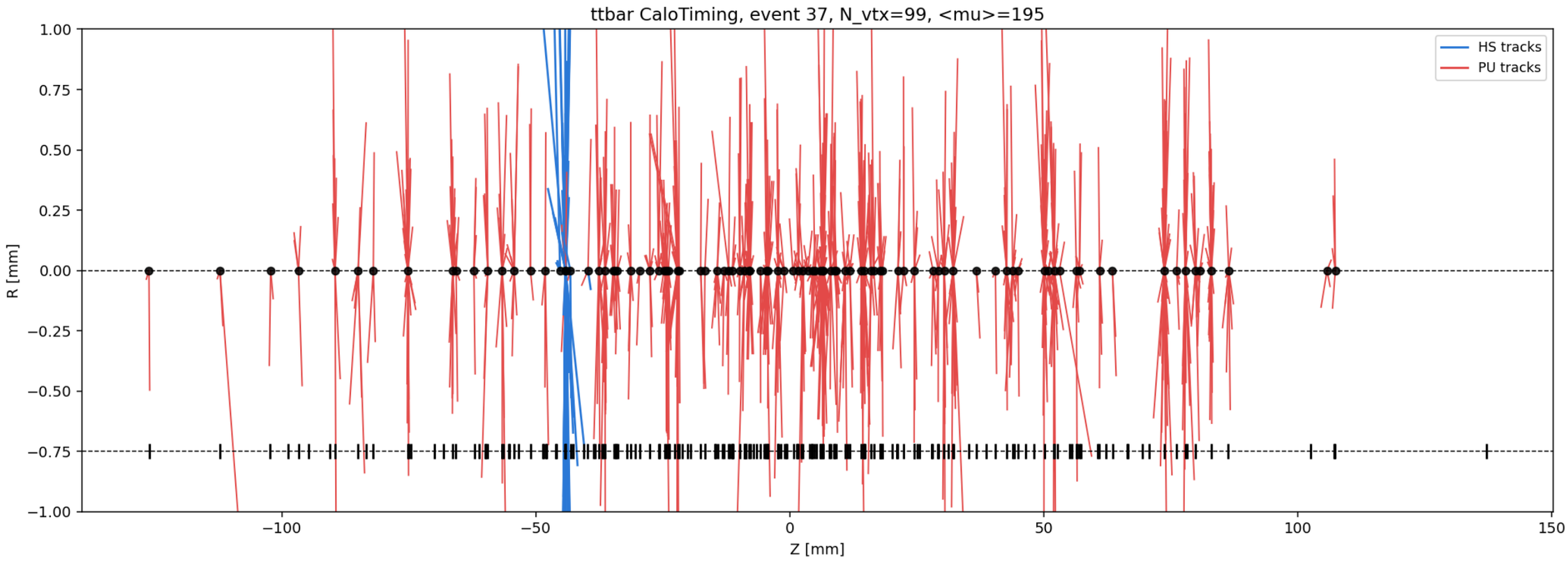
```
from tracehep.io.calotiming import load_vertex_event

vtx_event = load_vertex_event(
    "calo_timing_ntuple.root",
    event_index=37,
)
print(len(vtx_event.vertices), "vertices")
print(vtx_event.mu, "<mu>")
# -> 99 vertices
# -> 195.0 <mu>
```

The z-R vertex display, three styles

```
from tracehep import plot_vertices_zr
  plot_vertices_zr(vtx_event, style="plain").savefig("plain.png")
 plot_vertices_zr(vtx_event, style="styled").savefig("styled.png")
plot_vertices_zr(vtx_event, style="time_colored").savefig("time.png")
```

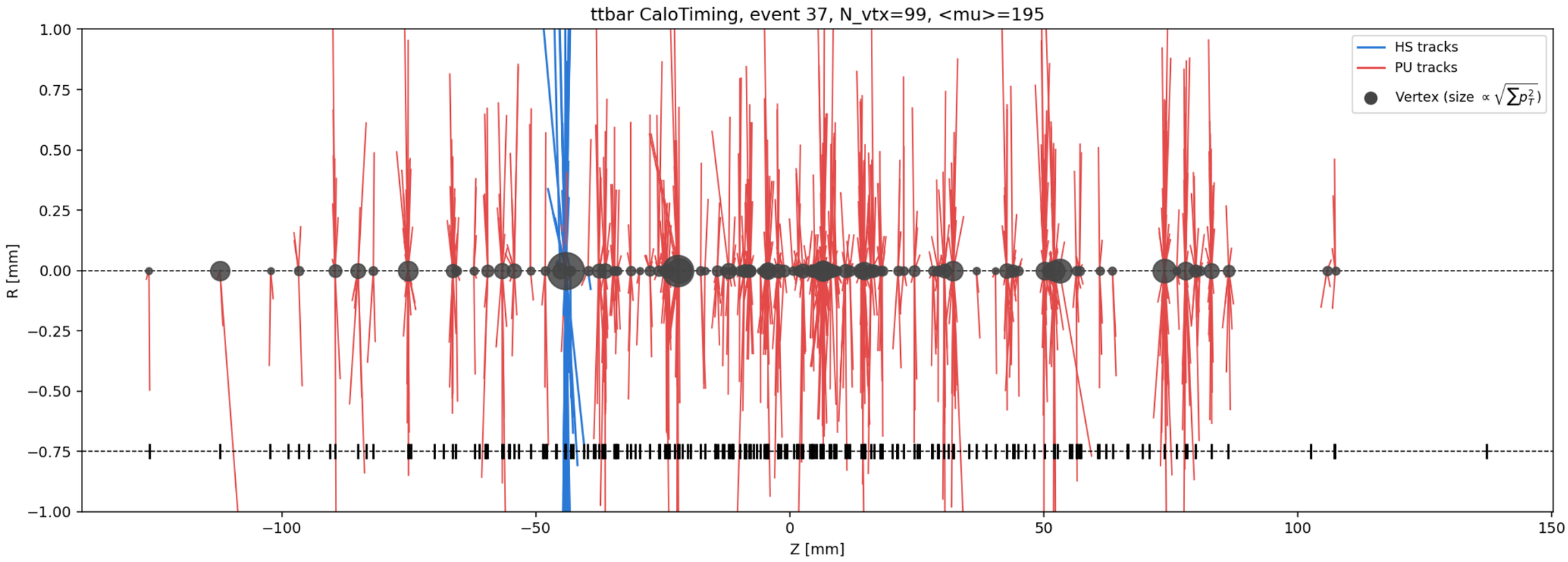
style is one of "plain", "styled", or "time_colored".

Output: style="plain"



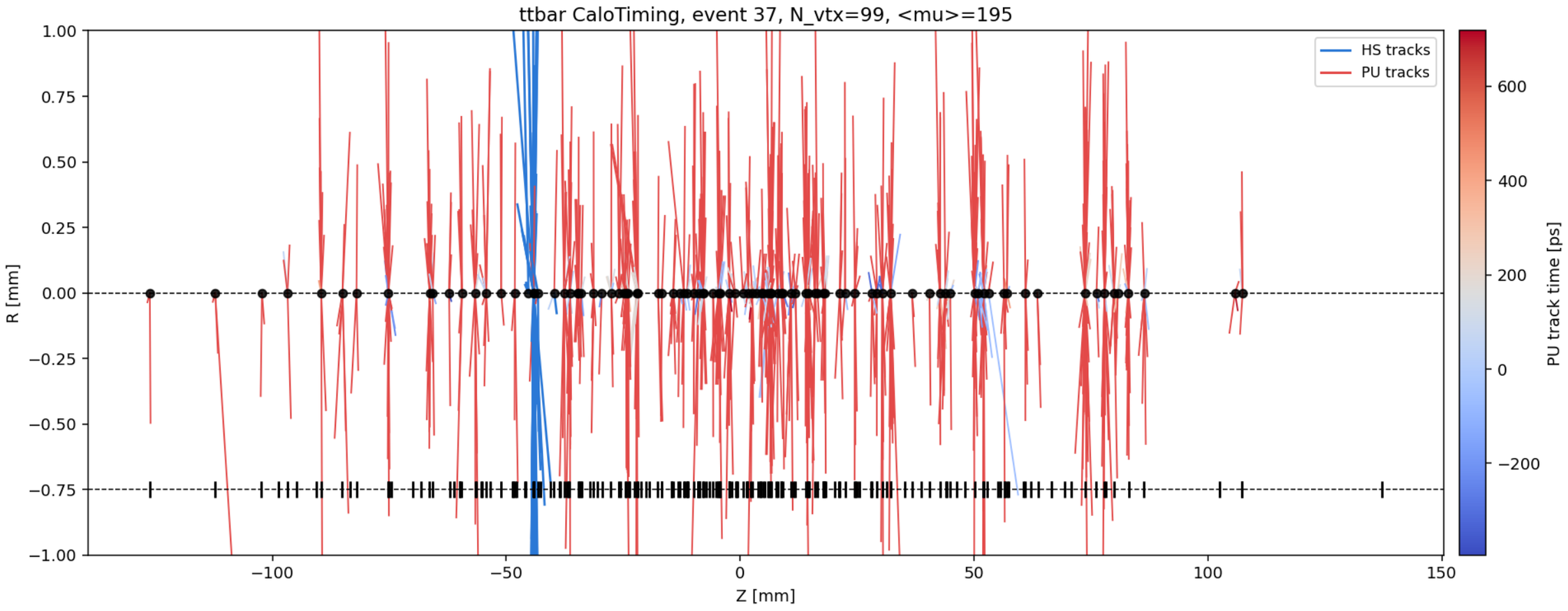
Flat HS (blue) / PU (red) track colouring, fixed vertex marker size

Output: style="styled"



Vertex marker size proportional to sqrt(sum pT^2); <mu> reported in the title

Output: style="time_colored"



PU tracks coloured by reconstructed track time (needs per-track timing)

Zooming into one vertex

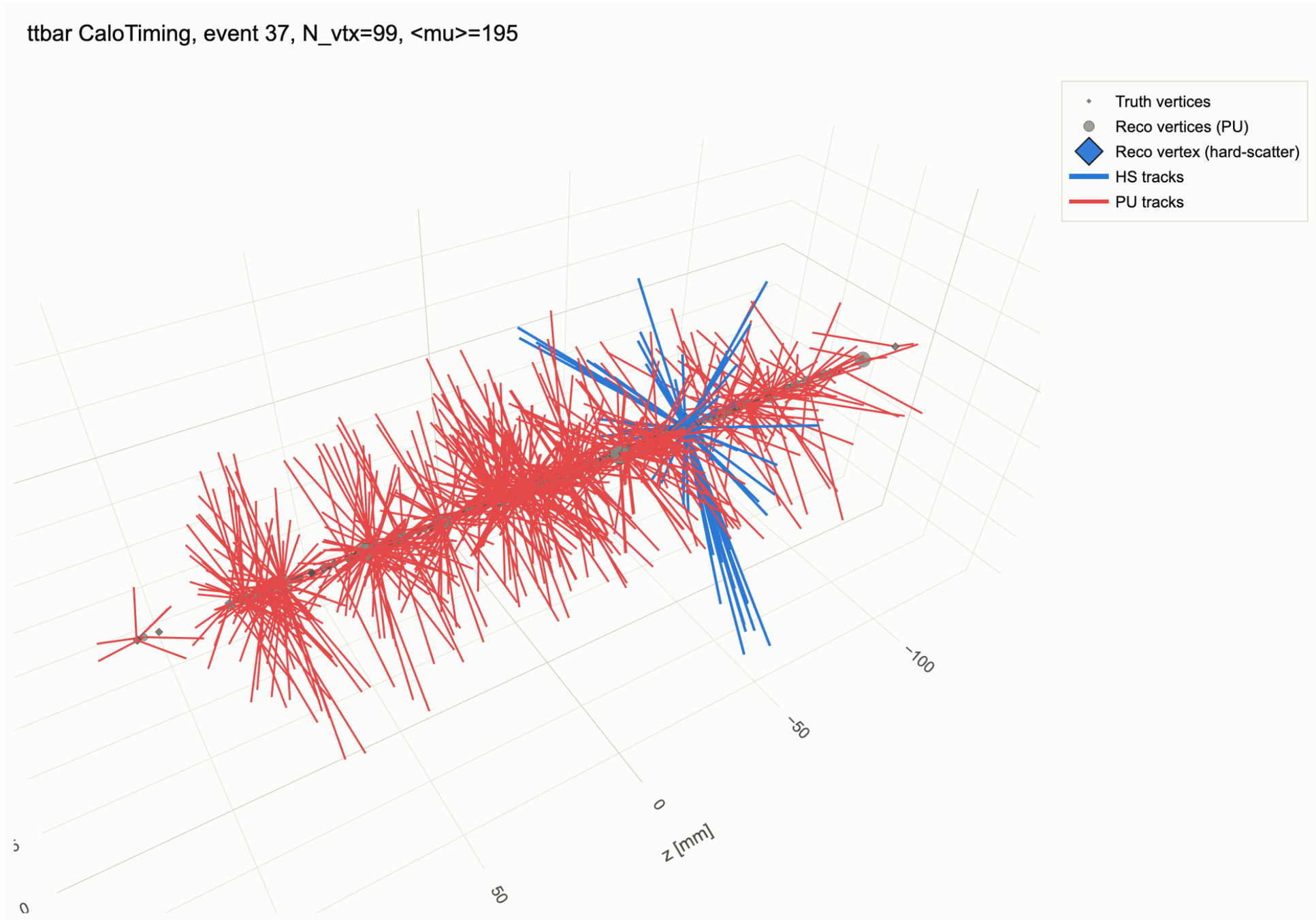
```
plot_vertices_zr(
    vtx_event, style="styled",
zoom_center=-44.2, zoom_range_mm=5.0,
    )
```

Same function -- pass zoom_center/zoom_range_mm to reproduce a single-vertex zoomed view.

Interactive 3D vertex display

```
from tracehep import plot_vertices_3d

  fig = plot_vertices_3d(vtx_event)
 fig.write_html("vertices_3d.html")
```

Output: the interactive 3D vertex view



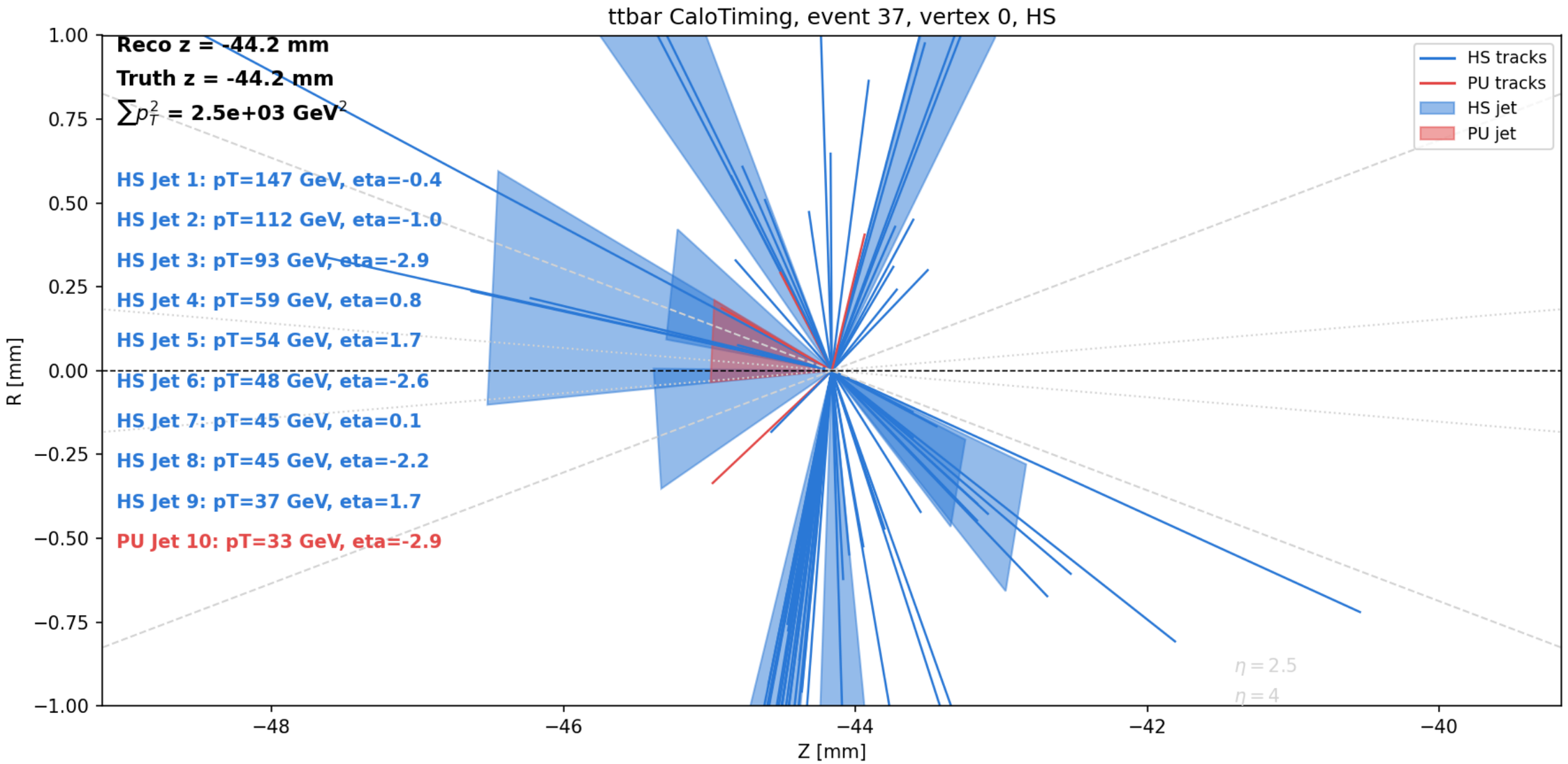
Every reconstructed and truth vertex at its real (x, y, z) position

One vertex, with its associated jets

```
from tracehep import plot_vertex_detail
from tracehep.io.calotiming import match_jets_to_vertex

vtx = vtx_event.vertices[0]
jets = match_jets_to_vertex(
"calo_timing_ntuple.root", event_index=37, vtx_z=vtx.z,
)

fig = plot_vertex_detail(vtx_event, vtx_index=0, jets=jets)
fig.savefig("vertex0_detail.png")
```

match_jets_to_vertex associates jets by Rpt (track-pT-fraction) matching -- bring your own jets for a different scheme.

Output: single-vertex detail with jets



Vertex 0 (HS): 10 jets matched (9 truth-HS, 1 truth-PU), sum(pT^2) = 2.5e3 GeV^2 -- tracks and jets each coloured by their own truth match

Filtering and jet collections

Real analyses rarely want every jet or every track --
trace-hep supports both, without touching a single plotting function:

1) Jet collection is a LOADER choice
-- which ROOT branches to read (Jet, GenJet, JetPUPPI, ...)

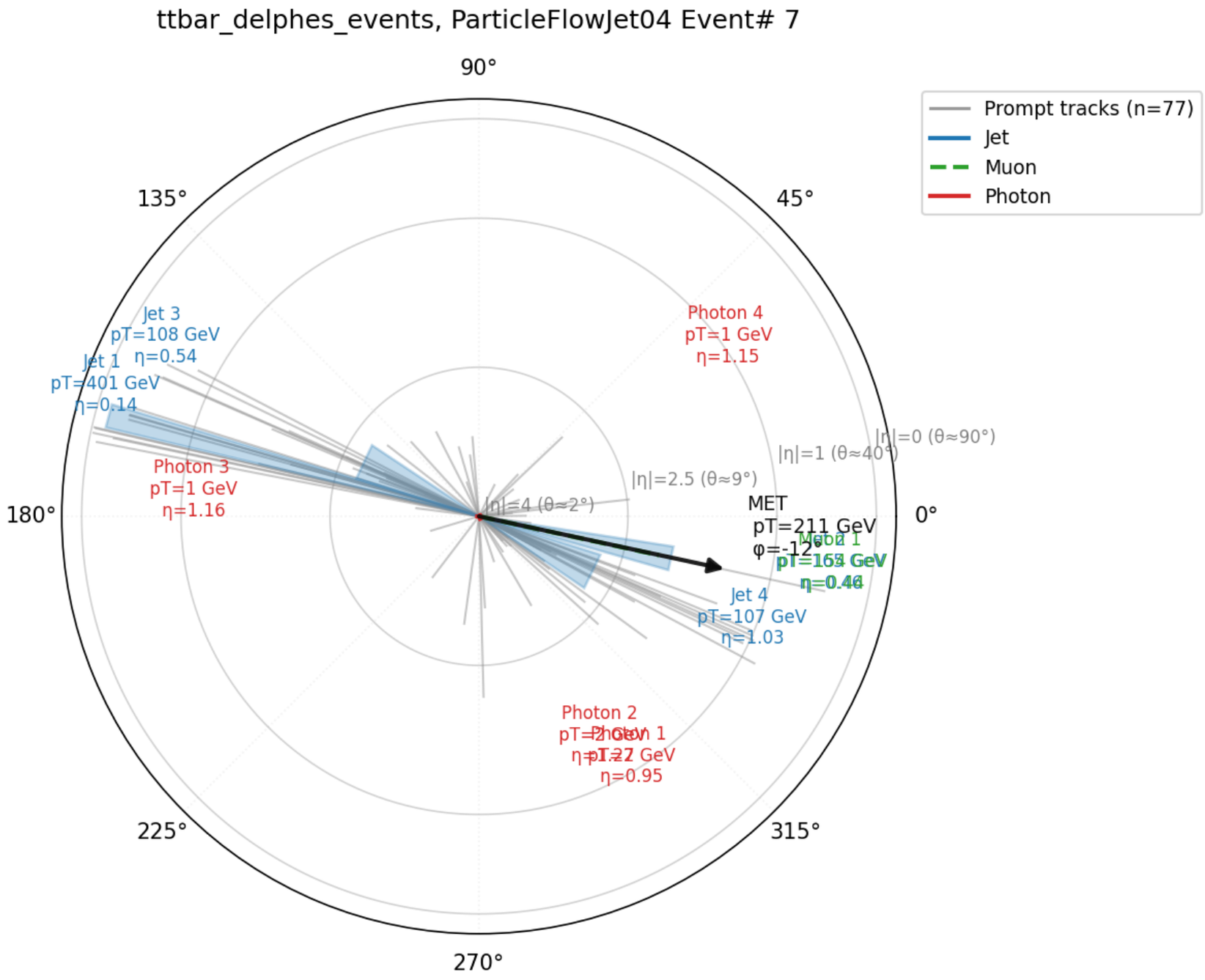
2) pT / eta cuts are a POST-LOAD, format-agnostic filter
-- tracehep.filters transforms an already-loaded Event or VertexEvent, so one cut applies identically to every display: polar, beam2d, 3D, z-R, vertex-detail, ...

Choosing a jet collection

```
from tracehep.io.delphes import load_events

# a Delphes card can define several jet collections --
   # default is "Jet"; read a different one by name
events = load_events(
"ttbar_delphes_events.root", indices=[7],
   jet_collection="ParticleFlowJet04",
                    )
trace.plot_event_polar(events[7], show_tracks=True)
```

Also works for match_jets_to_vertex (jet_collection="AntiKt4EMPFlowJets", ...) and the flat-ntuple loader (jet_branches=(...), bjet_branches=(...)).

Output: ParticleFlowJet04 vs. the default Jet collection



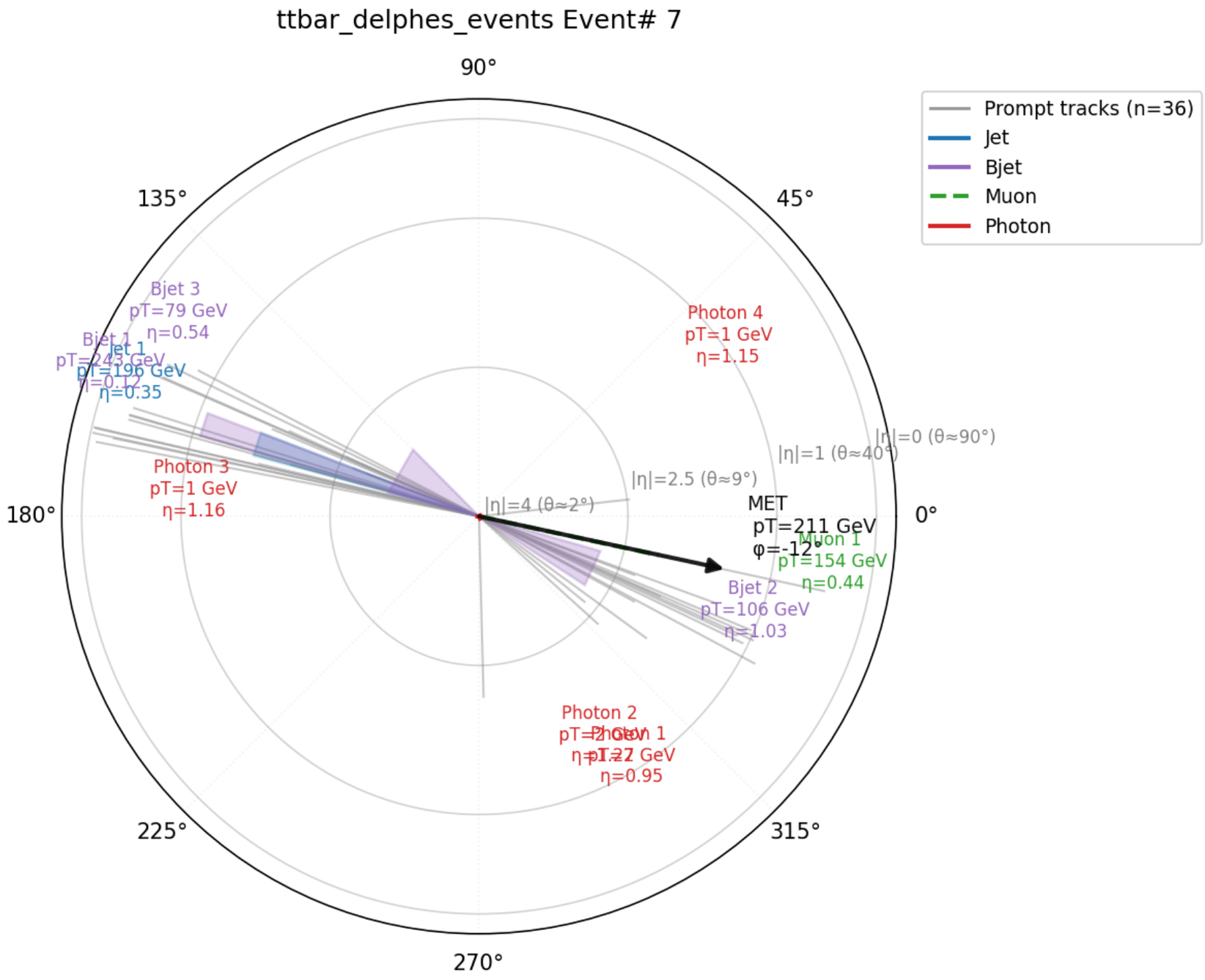
Same event, different jet collection: Jet 1 = 401 GeV here vs. 243 GeV with the default "Jet" collection -- a different clustering algorithm/input, same event

Cutting on pT and eta

```
from tracehep.filters import filter_event

event = load_events("ttbar_delphes_events.root", indices=[7])[7]

# keep only jets above 50 GeV within |eta| < 2.5,
# and tracks above 2 GeV within |eta| < 2.5
tight = filter_event(
    event,
    jet_pt_min=50.0, jet_eta_min=-2.5, jet_eta_max=2.5,
    track_pt_min=2.0, track_eta_min=-2.5, track_eta_max=2.5,
)
trace.plot_event_polar(tight, show_tracks=True)
```

eta_min/eta_max bound eta directly (signed, not abs(eta)) -- pass eta_min=0 to keep only positive-eta objects.

Output: after pT/eta cuts



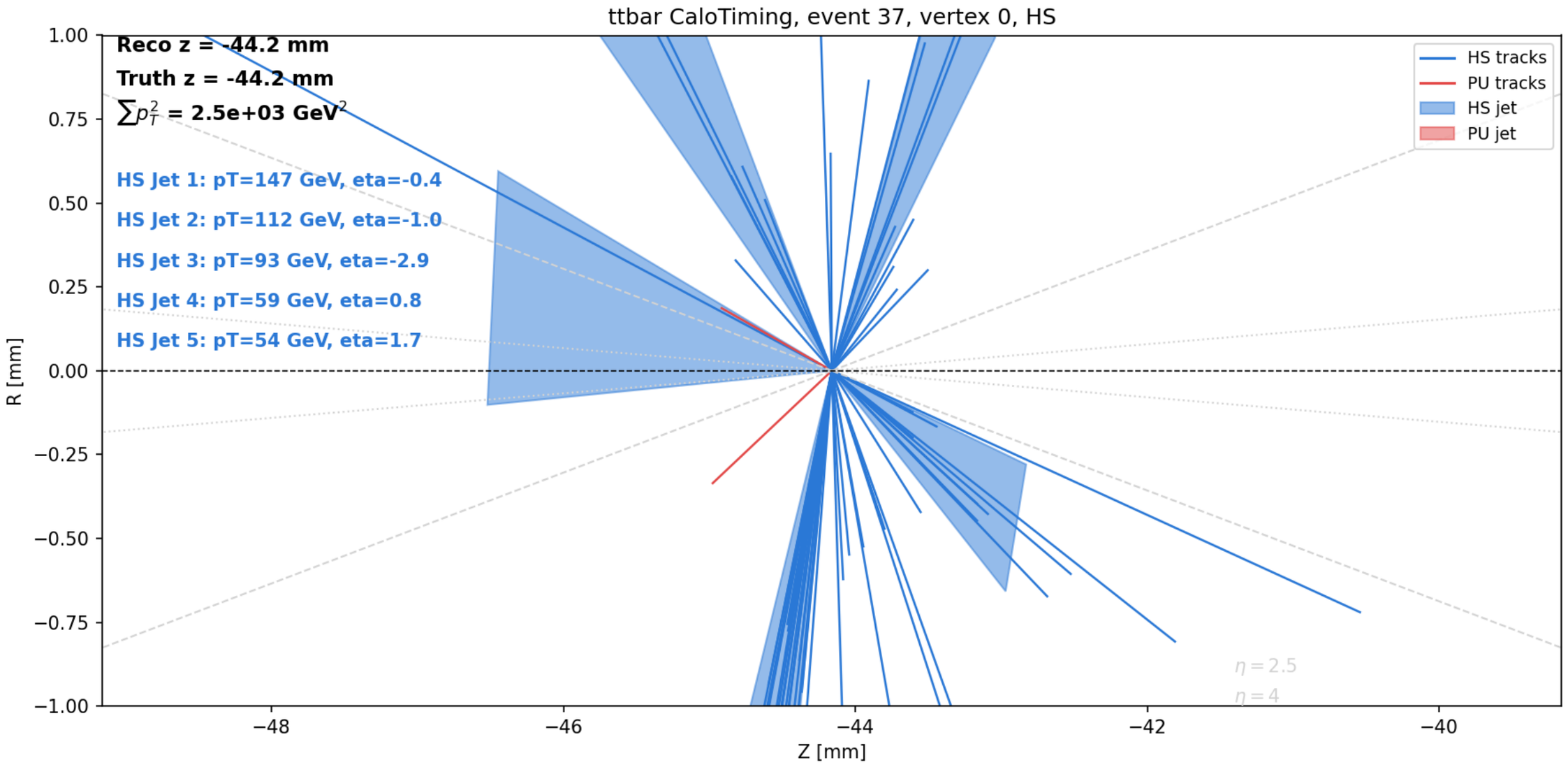
Same event 7: 77 -> 36 tracks after the pT/eta cut (all 4 jets already passed pT>50 GeV, |eta|<2.5)

Filtering a vertex scenario

```
from tracehep.filters import filter_vertex_event

vtx_event = load_vertex_event("calo_timing_ntuple.root", event_index=37)
vtx_event = filter_vertex_event(
vtx_event, track_pt_min=1.0, track_eta_min=-2.5, track_eta_max=2.5,
)
plot_vertices_zr(vtx_event, style="styled")
```

Vertices are never dropped, only the tracks fit to them -- each vertex's track_indices is remapped to stay consistent.

Output: vertex 0 after track pT/eta cuts



Event 37: 969 -> 607 tracks event-wide after the cut; vertex 0 goes from 10 to 5 matched jets once jet_pt_min=50 GeV is also applied

Filtering from the command line

```
trace-batch --input sample.root --indices 0 1 2 --outdir out/ \
                --jet-collection ParticleFlowJet04 \
          --jet-pt-min 30 --jet-eta-min -2.5 --jet-eta-max 2.5 \
                --show-tracks --track-pt-min 1.0 \
          --track-eta-min -2.5 --track-eta-max 2.5
```

Every filter available in Python is also a trace-batch flag -- no code needed for a batch of cut, collection-specific displays.

Real public data: ATLAS Open Data

tracehep.io.atlas_opendata reads the public ATLAS Open Data 13 TeV "mini-ntuple" format (opendata.atlas.cern) directly -- no Delphes, no private ntuple, no ROOT/CMSSW build needed.

The same flat schema is used for EVERY released 13 TeV dataset -- SM, Higgs, SUSY, exotic-resonance searches, ... -- so one loader works unchanged across all of them.

Find file URLs with the companion atlasopenmagic package rather than guessing paths by hand: pip install "trace-hep[opendata]"

Finding a dataset URL

```
import atlasopenmagic as atom

atom.set_release("2020e-13tev")
atom.available_keywords()      # -> ttbar, Higgs, Zprime, susy, ...

                               # ttbar, dilepton skim
urls = atom.get_urls("410000", skim="2lep", protocol="https")
```

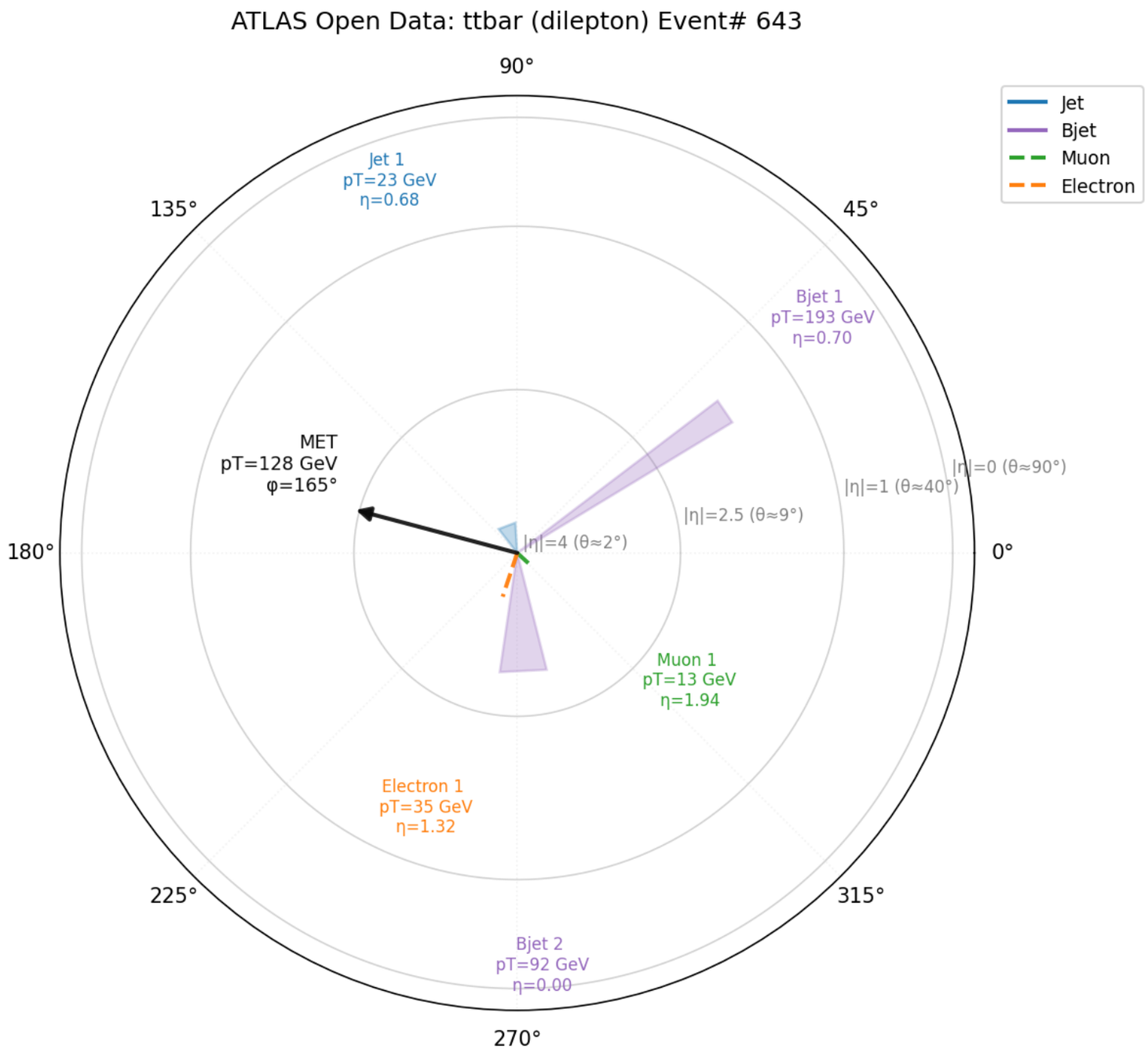
atlasopenmagic also covers 2024/2025 research-grade releases, not just the 2020 education set used here.

Loading and drawing an ATLAS Open Data event

```
from tracehep.io.atlas_opendata import load_events

events = load_events(
    "mc_410000.ttbar_lep.2lep.root", indices=[0, 1],
    label="ttbar (ATLAS Open Data)",
)
trace.plot_event_polar(events[1]).savefig("event1_polar.png")
```

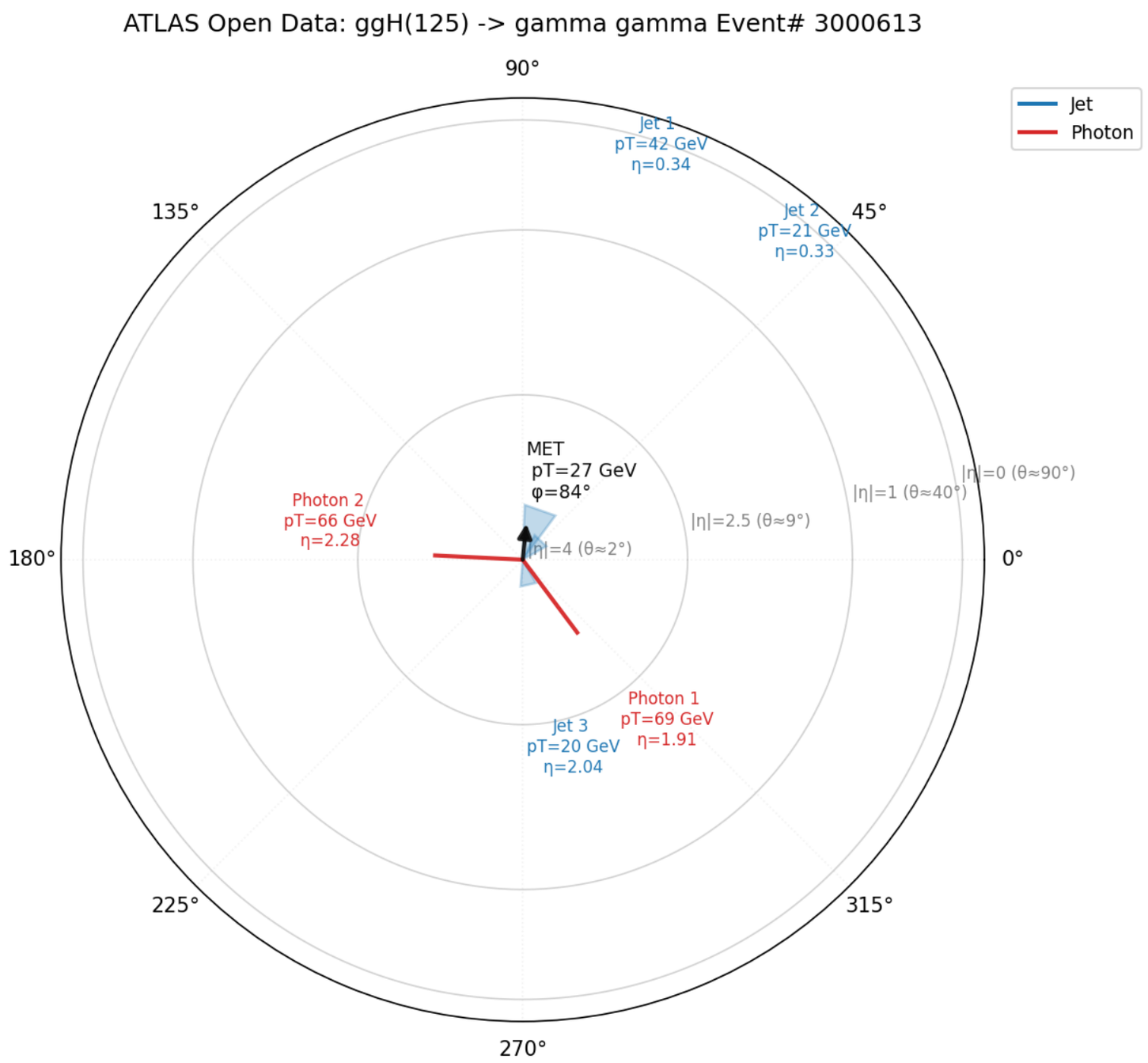
Momenta are stored in MeV in this ntuple -- the loader converts to GeV automatically.

Output: a real ttbar (e-mu) event



Genuine ATLAS Open Data dilepton ttbar candidate: 2 b-jets, one electron, one muon, and real MET -- no simulation shortcuts

Output: a real H -> gamma-gamma candidate



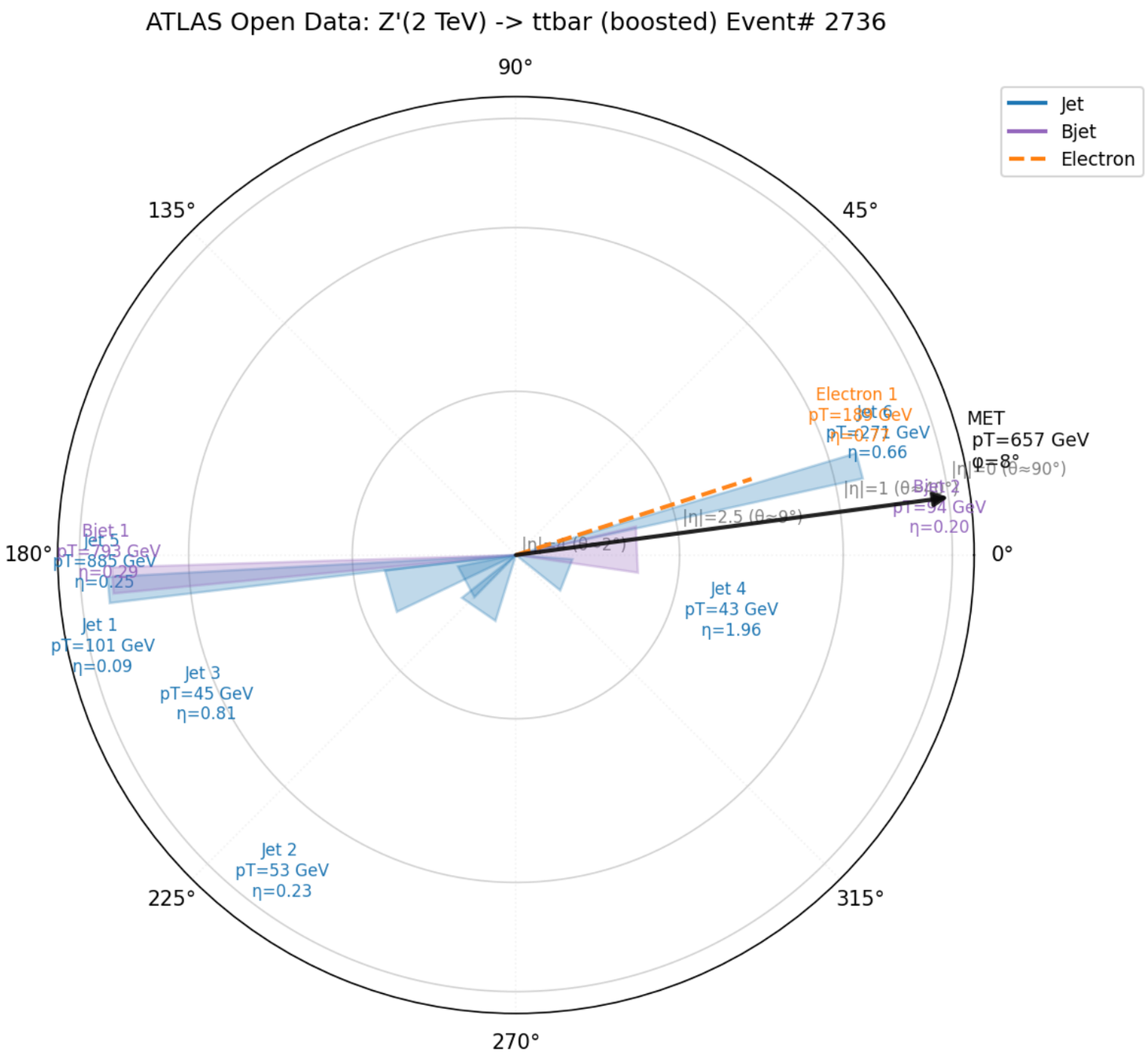
ggH(125) -> gamma-gamma skim: exactly two photons per event, plus the underlying-event jets

Boosted topologies: large-R jets

```
events = load_events(
"mc_301329.ZPrime2000_tt.1largeRjet1lep.root", indices=[4],
    include_large_r_jets=True,
label="Z'(2 TeV) -> ttbar (boosted)",
                )
trace.plot_event_polar(events[4])
```

include_large_r_jets=True appends the largeRjet_* collection -- b-tagging (MV2c10) is only defined for small-R jets.

Output: a boosted Z' -> ttbar event



2 TeV resonance, 657 GeV MET -- the large-R jet label overlaps its small-R constituents, as expected: they point the same direction by construction

Batch processing many events at once

```
from tracehep.io.delphes import load_events
from tracehep.batch import run_event_batch

events = load_events(path, indices=range(20))
run_event_batch(
    events, "out/", which=("polar", "beam2d"),
    polar_kwargs={"show_tracks": True},
)
# -> out/event0_polar.png, out/event0_beam2d.png, ...
```

Batch processing vertex scenarios

```
from tracehep.io.calotiming import load_vertex_event
from tracehep.batch import run_vertex_batch

vtx_events = {i: load_vertex_event(path, i) for i in range(10)}
run_vertex_batch(vtx_events, "out/", styles=("plain", "styled"))
  # -> out/vertices0_plain.png, out/vertices0_styled.png, ...
```

The trace-batch command-line tool

```
trace-batch \
--input ttbar_delphes_events.root \
  --indices 0 1 2 \
  --outdir out/ \
--which polar beam2d \
  --show-tracks \
  --label my_sample
```

No Python required -- useful for quick batch regeneration from a shell script.

Function reference: event displays

```
plot_event_polar(event, *, ax=None, eta_max=4.0, pt_scale=0.003,
                 eta_guides=(0,1,2.5,4.0), show_tracks=False,
            track_pt_scale=0.15, d0_displaced_mm=None, title=None)

     plot_event_beam2d(event, *, ax=None,
    eta_guides=(2.5,4.0), angle_scale=4.0, title=None)

   plot_event_3d(event, *, d0_displaced_mm=None,
           jet_half_angle_deg=9.0, title=None)
```

Function reference: vertex displays

```
plot_vertices_zr(vertex_event, *, style="plain",
        zoom_center=None, zoom_range_mm=None,
                ax=None, title=None)

plot_vertex_detail(vertex_event, vtx_index, jets=(),
        *, zoom_range_mm=5.0, ax=None, title=None)

plot_vertices_3d(vertex_event, *, title=None)

run_event_batch(events, outdir, *, which=(...),
            polar_kwargs=None, beam2d_kwargs=None)
run_vertex_batch(vertex_events, outdir, *, styles=(...))
```

Loader reference (tracehep.io) 1/2

```
delphes.load_event(path, index, with_tracks=True,
            jet_collection="Jet", label="")
delphes.load_events(path, indices, with_tracks=True,
            jet_collection="Jet", label="")

flat_ntuple.load_event_by_run_event(path, run, event,
tree_name="Ntuple", jet_branches=("JET_pt","JET_eta","JET_phi"),
bjet_branches=("bJET_pt","bJET_eta","bJET_phi"), label="")
flat_ntuple.load_events_by_run_event(path, pairs, ...)

calotiming.load_vertex_event(path, event_index,
            tree_name="ntuple", label="")
calotiming.match_jets_to_vertex(path, event_index, vtx_z,
*, jet_collection="AntiKt4EMTopoJets", track_idx_branch=None,
         jet_pt_min=30.0, rpt_min=0.02, sig_cut=3.0)

All four require: pip install "trace-hep[delphes]"
```

Loader reference (tracehep.io) 2/2

```
  atlas_opendata.load_event(path, index, tree_name="mini",
     btag_cut=0.6459, include_large_r_jets=False, label="")
atlas_opendata.load_events(path, indices, tree_name="mini",
     btag_cut=0.6459, include_large_r_jets=False, label="")

  Reads the public ATLAS Open Data 13 TeV mini-ntuple format
(opendata.atlas.cern) -- object-level only, no tracks/vertices.

        Requires: pip install "trace-hep[delphes]"
  URL discovery requires: pip install "trace-hep[opendata]"
```

Filter reference (tracehep.filters)

```
filter_event(event, *, jet_pt_min=None, jet_pt_max=None,
   jet_eta_min=None, jet_eta_max=None, track_pt_min=None,
 track_pt_max=None, track_eta_min=None, track_eta_max=None)

 filter_vertex_event(vertex_event, *, track_pt_min=None,
    track_pt_max=None, track_eta_min=None, track_eta_max=None)

     filter_jets(jets, *, pt_min=None, pt_max=None,
        eta_min=None, eta_max=None, btag_only=False)
   filter_tracks(tracks, *, pt_min=None, pt_max=None,
                 eta_min=None, eta_max=None)
```

All four return a filtered COPY -- the input is never modified.

Troubleshooting

* ImportError: uproot required
-> pip install "trace-hep[delphes]"

* ModuleNotFoundError: No module named tracehep (right after install)
-> from TestPyPI you need BOTH --index-url and
--extra-index-url (see Installation slides)

* KeyError from flat_ntuple loader
-> the (run, event) pair is not present in that file

* A track's 3D origin looks wrong from a calo-timing ntuple
-> that format has no per-track (x, y, z); origin defaults
to the owning vertex's (x, y) plus the track's own z0

Versioning and updates

* Published package versions are immutable
-> updating requires bumping the version and re-uploading

* Typical update loop:
edit code in src/tracehep/
add/update a test
bump version in pyproject.toml and __init__.py
python3 -m build
twine upload --repository-url https://test.pypi.org/legacy/ dist/*

Get involved

```
          Source:    github.com/wasikatcern/TRACE-HEP
Install:   pip install trace-hep  (TestPyPI now; PyPI planned)
                    License:   MIT
Tests:     pytest -q   (32 passing at v0.1.8)
                Contact:   wasikul.islam@cern.ch

Citing: accompanying paper citation to be added once posted.
```